

MyWebApp template

Contents

1. Overview	5
1.1. Technology Stack	7
1.2. Architecture	8
1.3. What's New in v0.8.3	8
1.4. What's New in v0.8.1	9
1.5. What's New in v0.8.0	9
1.6. What's New in v0.7.1	10
1.7. What's New in v0.7.0	10
1.8. What's New in v0.6.x	10
1.9. Earlier releases	10
2. Prerequisites	11
2.1. Required Software	11
2.2. Install Development Tools	11
2.3. Environment Setup	12
2.4. Secrets Management in Dev and in Prod	15
3. Initial Setup	16
3.1. 1. Clone the Repository	16
3.2. 2. Install Dependencies	16
3.3. 3. Create the Database	16
3.4. 4. Run Migrations	16
3.5. 5. Seed Demo Users	17

3.6. 6. Generate SQLC Code	17
3.7. 7. Build Tailwind CSS	17
3.8. 8. Run the Application	17
3.9. Quick Start (All-in-One)	18
4. Running with Docker and Docker Compose	19
4.1. Supplying <code>AGE_KEY</code> at container start	19
4.2. Using an untracked env file (recommended)	19
4.3. Docker Compose	20
4.4. <code>DB_HOST</code> reference	21
5. Running on Fly.io	22
6. Testing	23
6.1. Running the tests	23
6.2. How the test database works	23
6.3. Safety guards	23
6.4. Test coverage	24
7. Configuration Reference	25
7.1. Logging	25
7.2. Structured Log Fields	25
7.3. Rate Limiting	27
7.4. Gzip Compression	27
7.5. HTTP Security Headers	27
7.6. Secure Cookies	28
7.7. CSRF Protection	28
7.8. Authentication	28
7.9. Authorization	29
8. Versioning	30

8.1. Release procedure	30
9. Common Development Workflow	32
9.1. Daily Development	32
9.2. Database Changes	32
9.3. Adding New Queries	33
9.4. Adding a New API Endpoint	33
9.5. Adding a New HTMX Page	34
10. Project Structure	35
11. Project inventory for v0.8.3	37
12. Available Make Commands	39
12.1. Application	39
12.2. Dependencies	39
12.3. Database	40
12.4. Test database	40
12.5. Testing & Quality	40
12.6. Docker	41
12.7. Setup & Utilities	41
13. API Endpoints	43
13.1. Authentication API	43
13.2. Users API	43
13.3. Web (HTMX) Routes	43
13.4. Operational Endpoints	45
13.5. Example Requests	45
14. Troubleshooting	48
14.1. Common Issues	48
14.2. Getting Help	51

15. Contributing	53
15.1. Conventions.....	53
16. License	53
17. Contact	54

patrick toca v0.8.3, October 7th, 2026

Creation Date: September 6th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Overview

MyWebApp is a modern web application built with Go, using a clean architecture with PostgreSQL as the database. It combines a server-rendered HTMX frontend with a JSON API, sharing the same service layer.

The project leverages powerful tools like Gin for HTTP routing, SQLC for type-safe database queries, and [log/slog](#) for structured logging.

The application ships with a **public home page** reachable without authentication, a **read-only member directory** available to any authenticated user, and an **admin-only user-management surface** for creating and modifying accounts. The binary reports its own build metadata (version, commit, branch, build date) at runtime. A separate **test database environment** runs every DB-dependent test against an isolated PostgreSQL instance. A **Docker/Compose setup** runs the container locally against a host PostgreSQL, and a **Fly.io deployment** runs the same image in production with SOPS-encrypted secrets.

MyWebApp — sequence diagram

Anonymous visitor loads the Home page, signs in, and loads the admin user list.

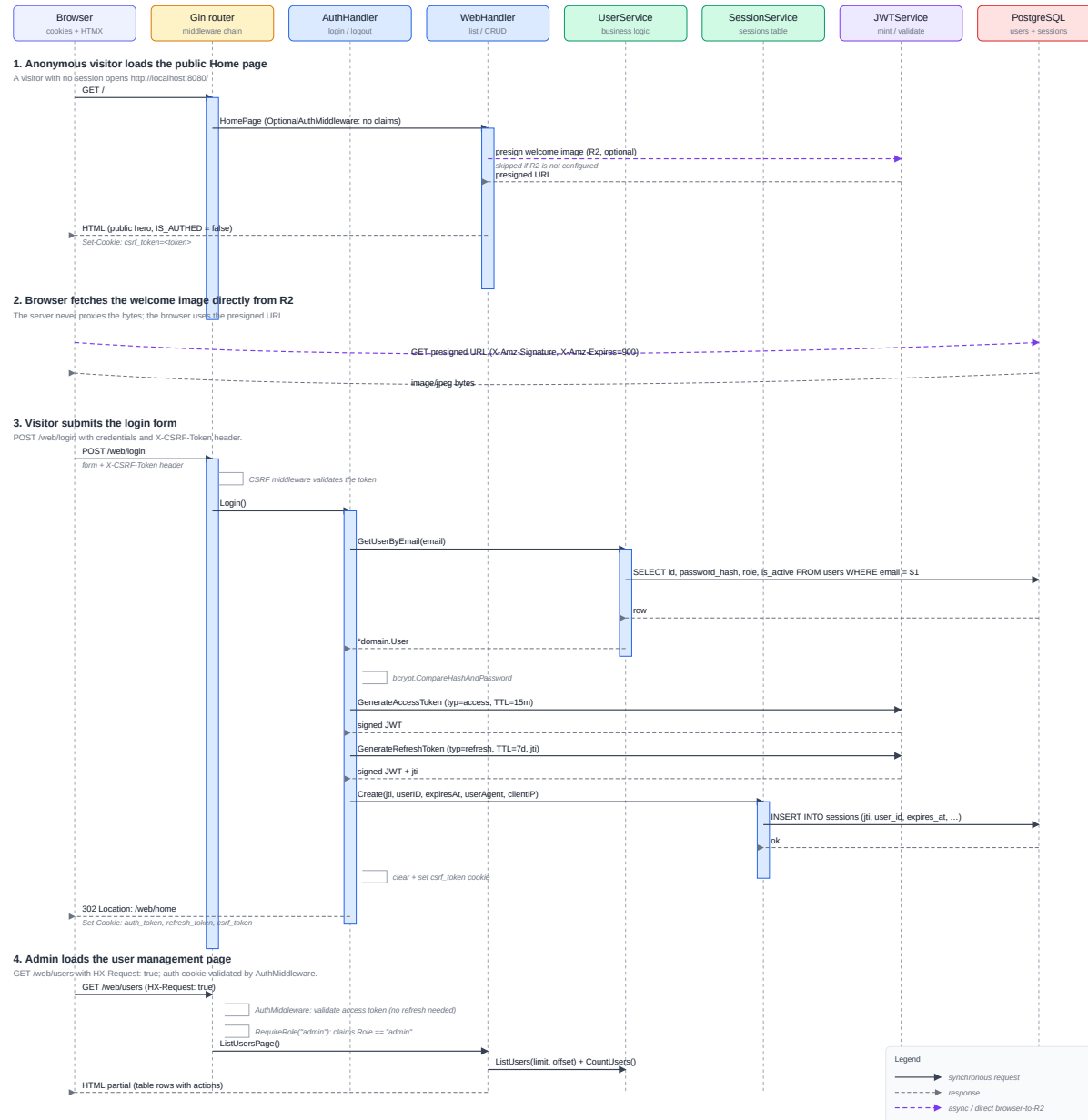


Figure 1. Login and admin-list sequence

1.1. Technology Stack

- **Go 1.21+** — Primary programming language
- **Gin** — HTTP web framework (with custom middleware for gzip, rate limiting, structured logging, security headers, and CSRF)
- **HTMX** — Hypermedia-driven frontend (no build step for interactivity)
- **Tailwind CSS** — Utility-first CSS framework
- **PostgreSQL** — Database
- **pgx/v5** — PostgreSQL driver and connection pool
- **SQLC** — Type-safe SQL code generation
- **log/slog** — Structured logging (JSON or text)
- **JWT** ([golang-jwt/jwt/v5](#)) — Access and refresh tokens with server-side revocation
- **bcrypt** ([golang.org/x/crypto](#)) — Password hashing
- **SOPS + age** — Encrypted secrets file, decrypted at boot by [internal/credentials](#)
- **Cloudflare R2** (optional) — Private welcome image via presigned URLs
- **Docker / Docker Compose** — Local containerised runs
- **Fly.io** — Production deployment with Fly Secrets
- **Air** — Live reload for development

1.2. Architecture

The application follows a clean architecture pattern:

- **Domain Layer** (`internal/domain/`) — Core business models, DTOs, and domain errors
- **Service Layer** (`internal/service/`) — Business logic, transaction orchestration, session management
- **Repository Layer** (`internal/repository/`) — Data access (SQLC-generated)
- **Handler Layer** (`internal/handler/`) — JSON API handlers
- **Web Layer** (`internal/web/`) — HTMX/HTML handlers, auth flows, cookie helpers, template helpers
- **Middleware Layer** (`internal/middleware/`) — Request ID, access log, gzip, rate limiting, security headers, CSRF, role check
- **Logging Layer** (`internal/logging/`) — Process-wide `slog` configuration
- **Auth Layer** (`internal/auth/`) — JWT issuance/validation, password hashing
- **Credentials Layer** (`internal/credentials/`) — Loads SOPS-encrypted secrets into the environment at boot
- **Version Layer** (`internal/version/`) — Build-time metadata injected by the linker
- **Testutil Layer** (`internal/testutil/`) — Isolated test database harness
- **Database Layer** (`pkg/db/`) — Pool configuration, transactions, `.env` loading
- **Object Storage** (`pkg/r2/`) — Cloudflare R2 client for presigned URLs

1.3. What's New in v0.8.3

- **Docker and Docker Compose verified end-to-end** — the image builds locally, connects to PostgreSQL, publishes port 8080, and `/health` returns `{"status":"ok"}`.
- **compose.yaml hardened** — prevents registry pulls, uses the working bridge network, preserves `host.docker.internal`, and uses a GET-compatible healthcheck.
- **.env.docker.local** now sets `PORT=8080`, and is gitignored alongside the other Docker env files.
- **SOPS + age secret loading** — `internal/credentials` decrypts `secrets.enc.yaml` at boot using `AGE_KEY`, and exports each known secret into

the process environment. See [the age/SOPS/Fly how-to](#).

- **Dockerfile, `fly.toml`, and logging updated** for the container workflow.
- **New doc:** [Running in Production with Docker and Fly.io](#).

1.4. What's New in v0.8.1

- **CSRF token signing** — the CSRF cookie now carries an HMAC signature over the token body. The middleware rejects any cookie whose signature does not verify, closing the cookie-tossing path in which an attacker plants a valid-looking cookie.
- **Bearer exemption tightened** — `CSRFMiddleware` skips validation only when `AuthMiddleware` has explicitly marked the request as Bearer-authenticated.
- **API paths accept only Bearer tokens** — cookie authentication is refused on `/api/*`.
- **Bearer token endpoint** — `POST /api/v1/auth/token` exchanges credentials for a Bearer access token.
- **Silent refresh reloads the user** — the middleware reads the current role and `is_active` flag from the database on every refresh.
- **Logout revokes the correct session** — `Logout` parses the refresh cookie to obtain the `jti` before revoking.
- **Rate limiter hardening** — keying uses Gin's `ClientIP()`; IPv6 is keyed by `/64`; the per-limiter map is capped with a fallback bucket; `Retry-After` reflects the actual rate; a `Stop` method prevents the cleanup goroutine from leaking.

1.5. What's New in v0.8.0

- **Isolated test database environment** — `internal/testutil` provides a per-binary test harness that resets the schema, applies migrations in-process, and seeds the demo users.
- **Expanded test coverage** — source-only coverage rose from 20.5% to 42.3%.
- **Bug fixes caught by the new tests** — `UserService.DeleteUser` returns `ErrUserNotFound` for a nonexistent user; gzip no longer advertises compression on 204 responses.

1.6. What's New in v0.7.1

- **Build-time version metadata** — `internal/version` carries the semantic version, git commit, branch, build date, and dirty flag, injected by `-ldflags -X`. A public `GET /version` endpoint reports the same information as JSON.
- **Release build tooling** — `scripts/build.sh` compiles with the correct `-ldflags` and `-trimpath` flags.
- **Stats endpoint available to all roles** — `/web/users/stats` moved out of the admin-only group.

1.7. What's New in v0.7.0

- **Public home page** — `/` and `/web/home` are served to anonymous visitors and authenticated users alike.
- **Read-only member directory** — `/web/members` lists registered users to every authenticated caller.
- **Unified post-login redirect** — every authenticated role lands on `/web/home` after login or registration.
- **Cache-Control hardening** — `abortUnauthenticated` emits `Cache-Control: no-store` on every 401 and login redirect.

1.8. What's New in v0.6.x

- **Role-based authorization** — `RequireRole` middleware, admin-only route groups, and dedicated role-change endpoints.
- **Self-service landing page** — `/web/me` for non-admin users.
- **Private R2 welcome image** — presigned URLs generated on demand.
- **Dedicated role-change forms** — separate DTO (`UpdateUserRoleRequest`) keeps `role` out of create/update payloads.

1.9. Earlier releases

- **v0.5.0** — HTTP security headers, secure cookies, CSRF protection, two-token JWT scheme with a `sessions` table, silent refresh.
- **v0.4.0** — HTTP security headers and secure cookie attributes.
- **v0.3.0** — Structured logging, request IDs, gzip, rate limiting, graceful shutdown, health endpoint.

2. Prerequisites

2.1. Required Software

- Go 1.21 or higher
- PostgreSQL 14 or higher
- Node.js 18+ (for Tailwind CSS build)
- Make (optional but recommended)
- Git
- Docker and Docker Compose (optional, for containerised runs)
- `age` and `sops` (for encrypted secrets — see [the age/SOPS/Fly how-to](#))
- `flyctl` (optional, for Fly.io deployment)

2.2. Install Development Tools

```
# Install Air (live reload)
go install github.com/air-verse/air@latest

# Install SQLC (type-safe SQL)
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest

# Install golang-migrate (migrations), with the PostgreSQL driver compiled in
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
```



The `-tags 'postgres'` flag is required. Without it, the `migrate` binary has no database drivers and will fail with `unknown driver postgres` (forgotten import?) on first use.

2.3. Environment Setup

Create a `.env` file in the project root:

```
# Server
PORT=8080
GIN_MODE=release

# Database
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=mywebapp
DB_SSLMODE=disable

# Pool tuning (optional – sensible defaults apply)
DB_MAX_CONNS=25
DB_MIN_CONNS=5
DB_MAX_CONN_LIFETIME=1h
DB_MAX_CONN_IDLE_TIME=30m
DB_HEALTH_CHECK_PERIOD=1m
DB_CONNECT_TIMEOUT=5s
DB_STATEMENT_TIMEOUT=30s

# -----
# Test database
# -----
# Tests run against a separate database, never against DB_NAME above.
# TEST_DB_NAME must not equal DB_NAME. If it is empty or matches,
# every DB-dependent test skips with a clear message.
TEST_DB_HOST=localhost
TEST_DB_PORT=5432
```

```
TEST_DB_USER=postgres
TEST_DB_PASSWORD=postgres
TEST_DB_NAME=mywebapp_test
TEST_DB_SSLMODE=disable

# Pool tuning for the test database (smaller than production).
TEST_DB_MAX_CONNS=10
TEST_DB_MIN_CONNS=1
TEST_DB_CONNECT_TIMEOUT=5s
TEST_DB_STATEMENT_TIMEOUT=30s

# Auth – ALWAYS override in production (or load from secrets.enc.yaml)
JWT_SECRET=change-me-in-production-please

# JWT token TTLs
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=168h

# Logging
LOG_FORMAT=json
LOG_LEVEL=info

# Security headers
CSP_REPORT_ONLY=true
CSP_REPORT_URI=
HSTS_MAX_AGE=0
HSTS_INCLUDE_SUBDOMAINS=false
HSTS_PRELOAD=false
X_FRAME_OPTIONS=DENY
REFERRER_POLICY=strict-origin-when-cross-origin
PERMISSIONS_POLICY=
COOP=same-origin
CORP=same-origin
```

```
# Secure cookies
COOKIE_SECURE=false
COOKIE_SAMESITE=lax
COOKIE_DOMAIN=

# CSRF
CSRF_FORM_FIELD=csrf_token
CSRF_HEADER_NAME=X-CSRF-Token
CSRF_TOKEN_LENGTH=32
CSRF_COOKIE_MAX_AGE=0
CSRF_SECRET=

# Cloudflare R2 (optional – leave all four empty to disable the welcome image)
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET_NAME=
R2_IMAGE_KEY=welcome.jpg
```



For local HTTP development, `COOKIE_SECURE=false` is mandatory. If left at the `true` default while running over plain HTTP, the browser accepts the cookie on login but silently refuses to send it back.

2.4. Secrets Management in Dev and in Prod

Secrets (`JWT_SECRET`, `CSRF_SECRET`, `DB_PASSWORD`, `TEST_DB_PASSWORD`, R2 credentials) are stored in a SOPS-encrypted `secrets.enc.yaml` and loaded at boot by `internal/credentials` using the `AGE_KEY` environment variable. In development, `AGE_KEY` is unset and the loader falls back to `~/.config/sops/age/keys.txt`; in containers and on Fly.io, it is supplied as a secret.

See [the age/SOPS/Fly how-to](#) for installation, key generation, editing the encrypted file, rotation, and troubleshooting.

3. Initial Setup

3.1. 1. Clone the Repository

```
git clone https://github.com/yourusername/mywebapp.git  
cd mywebapp
```

3.2. 2. Install Dependencies

```
make install  
# or manually:  
go mod download  
go mod tidy  
npm install
```

3.3. 3. Create the Database

```
createdb mywebapp  
# or:  
psql -c "CREATE DATABASE mywebapp;"
```

3.4. 4. Run Migrations

```
make migrate-up
```

This creates the `users` and `sessions` tables, along with their indexes and the foreign key from `sessions.user_id` to `users.id`.

3.5. 5. Seed Demo Users

```
make seed
```

This inserts five bcrypt-hashed demo users. The primary demo login is:

- **alice@example.com** / `password123` (admin)

3.6. 6. Generate SQLC Code

```
make generate
```

3.7. 7. Build Tailwind CSS

```
make build-css
```

3.8. 8. Run the Application

```
# Run normally
make run

# Or with live reload (recommended for development)
make dev
```

3.9. Quick Start (All-in-One)

```
make setup && make seed && make dev
```

4. Running with Docker and Docker Compose

The container image is self-contained. The two most common runtime failures are a missing `AGE_KEY` and a `DB_HOST` set to `localhost`. Full details are in [Running in Production with Docker and Fly.io](#).

4.1. Supplying `AGE_KEY` at container start

```
docker run --rm \
  -e AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" \
  -e DB_HOST=host.docker.internal \
  -e DB_PORT=5432 \
  -e DB_USER=pmjtoca \
  -e DB_NAME=MyWebApp \
  -e DB_SSLMODE=disable \
  --add-host host.docker.internal:host-gateway \
  -p 8080:8080 \
  mywebapp:debug
```

4.2. Using an untracked env file (recommended)

Create `.env.docker.local` with the runtime values:

```
AGE_KEY=AGE-SECRET-KEY-...
DB_HOST=host.docker.internal
DB_PORT=5432
DB_USER=pmjtoca
DB_NAME=MyWebApp
DB_SSLMODE=disable
PORT=8080
```

Then:

```
docker run --rm \
  --env-file .env.docker.local \
  --add-host host.docker.internal:host-gateway \
  -p 8080:8080 \
  mywebapp:debug
```

4.3. Docker Compose

The repository ships with a `compose.yaml` that:

- builds the image locally (no registry pull),
- uses a working bridge network,
- preserves `host.docker.internal:host-gateway`,
- uses a GET-compatible healthcheck,
- reads `.env.docker.local`.

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    env_file:
      - .env.docker.local
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

Run:

```
docker compose up --pull always
```

Verify:

```
curl -s http://localhost:8080/health  
# {"status":"ok", ...}
```

4.4. DB_HOST reference

Where PostgreSQL runs	Value for DB_HOST
On the host machine	host.docker.internal
In Docker Compose	The database service name, for example db
On Fly.io	The private or managed database hostname
Anywhere else	Its reachable DNS name or IP address

5. Running on Fly.io

```
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)"  
fly deploy
```

`AGE_KEY` is the only Fly Secret that needs to be set. Everything else lives in `secrets.enc.yaml`, which is decrypted at boot. Fly injects the secret into the running machine at start time, so it never appears in the image or in a repository.

6. Testing

The project ships with an isolated test database environment. Tests never touch the production database.

6.1. Running the tests

```
make test           # full suite, serialised (-p 1)
make test-coverage  # HTML report over every package
make test-coverage-src # report excluding generated/wiring packages
```

6.2. How the test database works

`internal/testutil` is the harness. Its `Setup()` function, called from each DB-dependent package's `TestMain`, does four things:

1. Opens a pool to the database named by `TEST_DB_*`.
2. Resets the schema (`DROP SCHEMA public CASCADE; CREATE SCHEMA public`).
3. Runs the migrations in-process.
4. Seeds the demo users.

6.3. Safety guards

- `TEST_DB_*` are separate variables from `DB_*`.
- `db.IsTestConfig` refuses to run if `TEST_DB_NAME` equals `DB_NAME`.
- If `TEST_DB_NAME` is empty or does not contain `test`, `ci`, `tmp`, or `scratch`, `LoadTestConfig` returns `(nil, false)` and every DB-dependent test skips.

6.4. Test coverage

Source-only coverage is approximately 58%. The per-package figures:

Package	Coverage
internal/auth	~78%
internal/handler	~62%
internal/middleware	~88%
internal/service	~60%
internal/version	~66%
internal/web	~70%

7. Configuration Reference

7.1. Logging

Variable	Values	Description
LOG_FORMAT	text json	Output format.
LOG_LEVEL	debug info warn error	Minimum level.

Logs are written to `logs/app.log`. When running in an interactive terminal, they are also mirrored to stderr.

7.2. Structured Log Fields

```
{
  "time": "2026-10-02T12:34:56.214266491+01:00",
  "level": "INFO",
  "msg": "http_request",
  "request_id": "a9346c63f1de4251ddf025d9043d453d",
  "method": "POST",
  "path": "/web/login",
  "status": 200,
  "latency": 2781319,
  "client_ip": "::1",
  "size": 2249
}
```

7.2.1. Request Correlation

Every request is assigned a `request_id`. The ID is returned as the `X-Request-ID` response header, attached to the request-scoped `*slog.Logger`, and included on every line emitted through `middleware.LoggerFromGin(c)`.

```
grep '"request_id":"a9346c63f1de4251ddf025d9043d453d"' logs/app.log
```

7.3. Rate Limiting

Three token-bucket limiters run by default:

Scope	Rate	Burst	TTL
Global (all routes)	20 req/s	40	10 min
Auth routes	5 req/min	5	15 min
Refresh route (<code>/web/refresh</code>)	1 req/s	30	15 min

Buckets are keyed by the client IP returned from Gin's `ClientIP()`, which honours the trusted-proxy configuration set on the engine.

7.4. Gzip Compression

The gzip middleware only compresses when `Accept-Encoding: gzip` is present, skips WebSocket/SSE upgrades, skips already-compressed types, sets `Vary: Accept-Encoding`, and removes `Content-Length`.

7.5. HTTP Security Headers

The `SecurityHeaders` middleware sets defence-in-depth headers on every response. Full rationale in `docs/security_headers_cookies_csrf_jwt_scheme.adoc`.

Header	Default
<code>Content-Security-Policy(-Report-Only)</code>	tailored to HTMX + Tailwind; report-only in dev
<code>Strict-Transport-Security</code>	disabled (<code>HSTS_MAX_AGE=0</code>)
<code>X-Frame-Options</code>	DENY
<code>X-Content-Type-Options</code>	nosniff
<code>Referrer-Policy</code>	strict-origin-when-cross-origin
<code>Permissions-Policy</code>	camera/mic/geo/etc. disabled

Header	Default
Cross-Origin-Opener-Policy	same-origin
Cross-Origin-Resource-Policy	same-origin

7.6. Secure Cookies

Cookie	HttpOnly	Purpose
auth_token	yes	Short-lived access token (JWT_ACCESS_TTL, default 15 min)
refresh_token	yes	Long-lived refresh token (JWT_REFRESH_TTL, default 7 days)
csrf_token	no	Readable by JavaScript; required for the double-submit pattern

7.7. CSRF Protection

Every mutating request (POST, PUT, PATCH, DELETE) carries a CSRF token that the server compares against the csrf_token cookie using a constant-time comparison. The cookie value also carries an HMAC signature; the middleware rejects any cookie whose signature does not verify under CSRF_SECRET.

7.8. Authentication

The application uses a **two-token JWT scheme**:

- **Access token** — short-lived (15 min default), presented on every request. Stateless.
- **Refresh token** — long-lived (7 days default), carries a jti claim recorded in the sessions table.

Two transports: **Cookie** (auth_token, refresh_token) for browser and HTMX requests; **Authorization: Bearer <access-token>** for API clients. Bearer requests are exempt from CSRF and are not eligible for silent refresh. Bearer tokens are issued by POST /api/v1/auth/token.

7.8.1. Silent Refresh

When a request arrives with an expired `auth_token` but a valid, unrevoked `refresh_token`, the middleware reloads the user from the database, mints a fresh access token, sets it as a new `auth_token` cookie, and lets the request proceed.

7.8.2. Revocation

Logout, role changes, and account deactivation all revoke the corresponding `sessions` row. A revoked refresh token cannot be exchanged for an access token, so the session ends within the access-token TTL.

7.9. Authorization

Three roles exist: `user`, `moderator`, and `admin`. Every route falls into one of three categories:

Category	Routes
Public — no authentication required	<code>/</code> , <code>/web/home</code> , <code>/web/login</code> , <code>/web/register</code> , <code>/web/refresh</code> , <code>/api/v1/auth/token</code> , <code>/health</code> , <code>/version</code> , <code>/static/*</code>
Authenticated — a valid session is required, any role	<code>/web/members</code> , <code>/web/members/partial</code> , <code>/web/me</code> , <code>/web/users/stats</code> , <code>/web/logout</code>
Admin-only — a valid session and <code>role=admin</code>	<code>/web/users</code> , <code>/web/users/by-id</code> , <code>/web/users/new</code> , <code>/web/users/:id/edit</code> , <code>/web/users/:id/role</code> , <code>/api/v1/users/*</code>

8. Versioning

The binary reports its own build metadata through two channels: the startup log line and the public `GET /version` endpoint.

```
curl -s http://localhost:8080/version | jq .
```

```
{
  "version": "v0.8.3",
  "commit": "9d4e6e335a06",
  "branch": "develop",
  "state": "v0.8.3",
  "built": "2026-10-07T10:39:10Z",
  "go": "1.23.0",
  "modified": false
}
```

8.1. Release procedure

Releases follow Git Flow. Set the version prefix once:

```
git config --global gitflow.prefix.versiontag v
```

Then, from `develop`:

```
git flow release start 0.8.3
# update the version lines in the AsciiDoc documents, add a
# "What's New in v0.8.3" section, rebuild the PDFs, commit
git flow release finish 0.8.3
```

```
git push origin main  
git push origin develop  
git push origin v0.8.3
```

Do **not** run `git push --tags` — push the single release tag explicitly so unrelated local tags are not published.

9. Common Development Workflow

9.1. Daily Development

```
make dev
```

9.2. Database Changes

9.2.1. 1. Create a Migration

```
make migrate-create name=add_user_avatar
```

9.2.2. 2. Apply / Rollback

```
make migrate-up      # apply
make migrate-down    # rollback one
make db-reset        # full reset (drop, up, seed)
```

9.2.3. Troubleshooting a Dirty Migration

```
migrate -path migrations -database "$DB_URL" force 0
psql -d mywebapp -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
make migrate-up
make seed
```

9.3. Adding New Queries

```
-- name: GetUserByEmailAndActive :one
SELECT id, email, username, full_name, is_active, role, last_login, created_at, updated_at
FROM users
WHERE email = $1 AND is_active = $2
LIMIT 1;
```

Then:

```
make generate
```

9.4. Adding a New API Endpoint

```
func (h *PostHandler) CreatePost(c *gin.Context) {
    logger := middleware.LoggerFromGin(c)

    var req domain.CreatePostRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    post, err := h.service.CreatePost(c.Request.Context(), req)
    if err != nil {
        logger.Error("api: CreatePost failed", "error", err)
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Something went wrong."})
        return
    }
}
```

```
c.JSON(http.StatusCreated, domain.ToPostResponse(post))  
}
```

9.5. Adding a New HTMX Page

```
func (h *WebHandler) ListPostsPage(c *gin.Context) {  
    posts, _ := h.service.ListPosts(c.Request.Context(), 20, 0)  
  
    data := gin.H{  
        "posts":      posts,  
        "csrf_token": middleware.TokenFromContext(c),  
    }  
  
    if IsHTMX(c) {  
        c.HTML(http.StatusOK, "post_list.html", data)  
        return  
    }  
    c.HTML(http.StatusOK, "base.html", gin.H{  
        "title":      "Posts",  
        "listEndpoint": "/web/posts",  
        "csrf_token":  middleware.TokenFromContext(c),  
    })  
}
```

10. Project Structure

```
mywebapp/
├── bin/                # Compiled binaries (gitignored)
├── cmd/
│   └── api/
│       └── main.go    # Entry point, middleware wiring
├── docs/
│   ├── images/
│   ├── age_sops_fly_howto.adoc
│   ├── security_headers_cookies_csrf_jwt_scheme.adoc
│   └── webapp_in_prod_with_docker_and_fly.adoc
├── internal/
│   ├── auth/          # JWT + bcrypt
│   ├── credentials/   # SOPS/age loader
│   ├── domain/        # Models, DTOs, errors
│   ├── handler/       # JSON API handlers
│   ├── logging/       # slog setup
│   ├── middleware/    # accesslog, csrf, gzip, ratelimit, requestid, require_role, security_headers
│   ├── repository/    # SQLC-generated
│   ├── service/       # business logic
│   ├── templates/     # HTMX templates + partials
│   ├── testutil/      # isolated test DB harness
│   ├── version/       # build metadata
│   └── web/           # HTMX handlers, cookies, auth
├── logs/
│   └── app.log        # gitignored
├── migrations/
├── pkg/
│   ├── db/            # pool config, tx helper
│   └── r2/            # Cloudflare R2 presigned URLs
├── scripts/
└── build.sh
```

```
|   └── seed.go
|   ├── sqlc/
|   ├── static/
|   ├── .env                # gitignored
|   ├── .env.docker.local  # gitignored
|   ├── .air.toml
|   ├── .gitignore
|   ├── compose.yaml       # Docker Compose
|   ├── Dockerfile
|   ├── fly.toml
|   ├── Makefile
|   ├── go.mod
|   ├── go.sum
|   ├── package.json
|   ├── tailwind.config.js
|   ├── secrets.enc.yaml   # SOPS-encrypted secrets
|   └── README.adoc        # This file
```

11. Project inventory for v0.8.3

SCC Report — HTML Output

Source Count & Complexity analysis across all languages

128 Files

24,233 Lines

18,374 Code

2,532 Complexity

Language	Files	Lines	Blank	Comment	Code	Complexity	Bytes	Uloc	Code %	Bytes %	Cplx %
Go	73	13,009	1,709	1,649	9,651	1,865	376,717	0	52.5%	41.9%	73.7%
HTML	16	1,869	131	28	1,710	0	92,900	0	9.3%	10.3%	0.0%
Shell	10	933	127	138	668	54	30,630	0	3.6%	3.4%	2.1%
SQL	6	190	28	26	136	6	6,026	0	0.7%	0.7%	0.2%
AsciiDoc	5	5,913	1,312	0	4,601	0	188,494	0	25.0%	21.0%	0.0%
SVG	5	1,155	192	181	782	2	111,710	0	4.3%	12.4%	0.1%
YAML	4	134	1	38	95	0	7,999	0	0.5%	0.9%	0.0%
JavaScript	2	27	0	1	26	539	48,325	0	0.1%	5.4%	21.3%
TOML	2	123	17	35	71	0	3,463	0	0.4%	0.4%	0.0%
CSS	1	51	1	0	50	0	1,473	0	0.3%	0.2%	0.0%
Docker ignore	1	36	5	6	25	0	459	0	0.1%	0.1%	0.0%
Dockerfile	1	78	16	32	30	2	2,575	0	0.2%	0.3%	0.1%
JSON	1	15	0	0	15	0	520	0	0.1%	0.1%	0.0%
Makefile	1	700	81	105	514	64	27,131	0	2.8%	3.0%	2.5%
Total	128	24,233	3,620	2,239	18,374	2,532	898,422	0	100%	100%	100%



Estimated Cost to Develop (organic)
\$574,189



Estimated Schedule Effort
11.14 months



Estimated People Required
4.58

Code Distribution (top languages)



Go — 52.5%

HTML — 9.3%

AsciiDoc — 25.0%

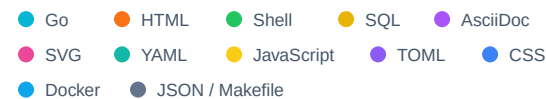


Figure 2. Source code statistics for MyWebApp v0.8.3

12. Available Make Commands

12.1. Application

Command	Description
<code>make run</code>	Run the application in the foreground
<code>make start</code>	Build and start in the background (writes <code>.pid</code>)
<code>make stop</code>	Stop the background process
<code>make restart</code>	Stop + start
<code>make status</code>	Show whether the app is running
<code>make logs</code>	Tail <code>logs/app.log</code>
<code>make dev</code>	Live reload (Air) + Tailwind watch
<code>make build</code>	Build Tailwind + production binary
<code>make build-css</code>	Build Tailwind CSS only
<code>make watch-css</code>	Tailwind in watch mode
<code>make version</code>	Print the version metadata that <code>make build</code> will inject
<code>make version-run</code>	Query <code>/version</code> on the running server

12.2. Dependencies

Command	Description
<code>make install</code>	Go + Node + dev tools
<code>make generate</code>	Regenerate SQLC code

12.3. Database

Command	Description
<code>make migrate-up</code>	Apply migrations
<code>make migrate-down</code>	Roll back one migration
<code>make migrate-create name=<name></code>	Create a new migration pair
<code>make seed</code>	Seed demo users (bcrypt-hashed)
<code>make db-reset</code>	stop → migrate-down → migrate-up → seed
<code>make db-shell</code>	Open <code>psql</code>
<code>make db-check</code>	Verify connection
<code>make db-tables</code>	List tables

12.4. Test database

Command	Description
<code>make test-db-create</code>	Create the test database (idempotent)
<code>make test-db-drop</code>	Drop the test database (with confirmation)
<code>make test-db-reset</code>	Drop and recreate the test database
<code>make test-db-check</code>	Print the resolved test configuration
<code>make test-db-shell</code>	Open <code>psql</code> against the test database

12.5. Testing & Quality

Command	Description
---------	-------------

<code>make test</code>	Run all tests (serialised; uses <code>TEST_DB_*</code>)
<code>make test-fast</code>	Run non-DB packages in parallel
<code>make test-coverage</code>	Raw HTML coverage over every package
<code>make test-coverage-src</code>	Source-only coverage
<code>make fmt</code>	<code>go fmt</code> + CSS rebuild
<code>make lint</code>	<code>golangci-lint</code> (auto-installs if missing)

12.6. Docker

Command	Description
<code>make docker-up</code>	Start containers
<code>make docker-down</code>	Stop containers
<code>make docker-build</code>	Build images
<code>make docker-logs</code>	Follow container logs

12.7. Setup & Utilities

Command	Description
<code>make setup</code>	install → generate → migrate-up → build-css
<code>make dev-setup</code>	Same as <code>setup</code> , hints at <code>make dev</code>
<code>make all</code>	clean → install → generate → migrate-up → build-css → build
<code>make clean</code>	Remove binaries, <code>node_modules</code> , CSS output
<code>make check-env</code>	Print loaded environment
<code>make info</code>	Project metadata

make dirs	Create standard directory layout
-----------	----------------------------------

13. API Endpoints

13.1. Authentication API

Method	Endpoint	Description
POST	<code>/api/v1/auth/token</code>	Exchange credentials for a Bearer access token

13.2. Users API

Method	Endpoint	Description
POST	<code>/api/v1/users</code>	Create a new user
GET	<code>/api/v1/users</code>	List users (paginated)
GET	<code>/api/v1/users/:id</code>	Get user by ID
PUT	<code>/api/v1/users/:id</code>	Update user
PUT	<code>/api/v1/users/:id/role</code>	Change a user's role
DELETE	<code>/api/v1/users/:id</code>	Delete user

All Users API routes require `Authorization: Bearer <access-token>` and `role=admin`. Cookie authentication is refused on `/api/*`.

13.3. Web (HTMX) Routes

Method	Endpoint	Description
GET	<code>/</code>	Public home page
GET	<code>/web/home</code>	Public home page (explicit URL)

GET	/web/members	Member directory (authenticated)
GET	/web/members/partial	Member directory HTMX fragment
GET	/web/login	Login page / form partial
POST	/web/login	Submit login
GET	/web/register	Register page / form partial
POST	/web/register	Submit registration
POST	/web/refresh	Exchange refresh token for a new access token
POST	/web/logout	Revoke the session and clear all cookies
GET	/web/me	Self-service account page
GET	/web/users	Users list (admin)
GET	/web/users/by-id	Users list by ID (admin)
GET	/web/users/new	New user form (admin)
POST	/web/users	Create user (admin)
GET	/web/users/:id/edit	Edit user form (admin)
PUT	/web/users/:id	Update user (admin)
GET	/web/users/:id/role	Role-change form (admin)
PUT	/web/users/:id/role	Submit role change (admin)
DELETE	/web/users/:id	Delete user (admin)
GET	/web/users/stats	Live member counts (any authenticated role)

13.4. Operational Endpoints

Method	Endpoint	Description
GET	/health	DB ping + pgx pool statistics
GET	/version	Build metadata as JSON
GET	/test	Template smoke test
GET	/static/*	Static assets

13.5. Example Requests

13.5.1. API token (JSON)

```
POST /api/v1/auth/token HTTP/1.1
Content-Type: application/json
```

```
{
  "email": "alice@example.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 900
}
```

13.5.2. Health Check

```
GET /health HTTP/1.1
```

```
{
  "status": "ok",
  "time": 1758567833,
  "pool": {
    "total": 5,
    "idle": 5,
    "acquired": 0,
    "max": 25,
    "constructing": 0,
    "acquire_count": 12,
    "empty_acquire": 0,
    "canceled_acquire": 0,
    "acquire_duration_ms": 3
  }
}
```

13.5.3. Version

```
GET /version HTTP/1.1
```

```
{
  "version": "v0.8.3",
  "commit": "9d4e6e335a06",
  "branch": "develop",
  "state": "v0.8.3",
}
```

```
"built": "2026-10-07T10:39:10Z",  
"go": "1.23.0",  
"modified": false  
}
```

14. Troubleshooting

14.1. Common Issues

14.1.1. Database Connection Failed

```
pg_isready  
make check-env  
make db-check
```

Inside a container, `localhost` refers to the container itself. Set `DB_HOST=host.docker.internal` (with `--add-host host.docker.internal:host-gateway` on Linux).

14.1.2. Missing `AGE_KEY`

The application fails fast when `AGE_KEY` is absent. Supply it at runtime via `--env-file`, `-e`, or a Fly Secret. Never bake it into the image or commit it. See [the Docker/Fly doc](#).

14.1.3. SQLC Generation Fails

```
cd sqlc && sqlc generate  
# or:  
make generate
```

14.1.4. Migration Fails

```
migrate -path migrations -database "$DB_URL" version
```

```
migrate -path migrations -database "$DB_URL" force <version>
```

For a development database:

```
psql -d mywebapp -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"  
make migrate-up  
make seed
```

14.1.5. migrate: command not found

```
export PATH="$HOME/go/bin:$PATH"  
echo 'export PATH="$HOME/go/bin:$PATH"' >> ~/.zshrc
```

14.1.6. unknown driver postgres (forgotten import?)

```
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
```

14.1.7. no migration found for version 0

```
psql -d mywebapp -c "DELETE FROM schema_migrations;"  
make migrate-up
```

14.1.8. Air Not Found

```
make install
```

14.1.9. Rate Limited During Development

Increase the global limiter in `cmd/api/main.go`:

```
globalLimiter := middleware.NewRateLimiter(60, 120, 10*time.Minute, 100_000)
```

14.1.10. Logged Out Immediately After Login

Set `COOKIE_SECURE=false` in `.env` for local dev.

14.1.11. Chrome Serves a Stale Redirect for /

Clear site data for `localhost` in DevTools → Application → Storage → **Clear site data**.

14.1.12. CSRF 403 on a Form Submission

Reload the page and resubmit. In a cluster, set `CSRF_SECRET` to the same value on every node.

14.1.13. DB-dependent tests skip

```
make test-db-check  
make test-db-create
```

14.1.14. multiple definitions of TestMain

```
grep -rn 'func TestMain' internal/
```

Only one match per package is correct.

14.1.15. Logs Are JSON When I Want Text

```
LOG_FORMAT=text
```

14.1.16. I Want Debug-Level Logs

```
LOG_LEVEL=debug
```

14.2. Getting Help

- Tail the structured logs: `make logs`
- Grep one request: `grep '"request_id":' logs/app.log`
- Check the health endpoint: `curl localhost:8080/health | jq`
- Check the running version: `curl localhost:8080/version | jq`
- Inspect active sessions:

```
psql -d mywebapp -c "SELECT jti, user_id, issued_at, expires_at, revoked_at FROM sessions ORDER BY id DESC LIMIT 10;"
```

- Official docs:
 - <https://gin-gonic.com/>
 - <https://htmx.org/>
 - <https://sqlc.dev/>
 - <https://github.com/golang-migrate/migrate>
 - <https://github.com/air-verse/air>

- <https://pkg.go.dev/log/slog>
- <https://github.com/golang-jwt/jwt>
- <https://getsops.io/>
- <https://age-encryption.org/>

15. Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Run `make test` and `make lint`
5. Submit a pull request

15.1. Conventions

- Use `middleware.LoggerFromGin(c)` (not `slog.Default()`) inside handlers.
- Return domain sentinel errors (`domain.ErrUserNotFound`, etc.) from services; map them in handlers.
- New SQL goes in `sqlc/queries.sql` or `sqlc/queries_sessions.sql`; never hand-edit `internal/repository/*.go`.
- New HTMX pages should expose a partial template and use a stable container id.
- Pass `csrf_token` into template data for any page rendering a mutating form.
- Mutating HTMX requests get `X-CSRF-Token` injected automatically.
- The root route `/` and `/web/home` are public. Any new protected route belongs behind `authMW`, and any admin-only route behind `RequireRole("admin")`.
- The public Home page uses `OptionalAuthMiddleware`. Do not swap in `authMW` on that route.
- New DB-dependent tests should call `testutil.NewPool(t)`, never `db.LoadConfig()` or `db.NewPool()`.
- Secrets belong in `secrets.enc.yaml`, never in `.env`, `fly.toml`, or a committed env file.

16. License

This project is licensed under the MIT License — see the LICENSE file for details.

17. Contact

- Author: patrick toca
- Email: pmjtoca@gmail.com
- Issue Tracker: <https://github.com/yourusername/mywebapp/issues>
- Repository: <https://github.com/yourusername/mywebapp>